



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления» (ИУ)

КАФЕДРА «Информационная безопасность» (ИУ8)

ТЕОРИЯ ИГР И ИССЛЕДОВАНИЕ ОПЕРАЦИЙ

Лабораторная работа № 4

на тему:

«Решение позиционной игры методом обратной индукции»

Вариант 13

Студент:

ИУ8-105
(Группа)

Б. Е. Шекунов
(И. О. Фамилия)

Преподаватель:

Н.С. Коннова
(И.О. Фамилия)

Москва, 2026 г.

Цель работы:

Изучить метод обратной индукции и его применение к решению конечных позиционных игр с полной информацией. Изучить свойства решений таких игр.

Постановка задачи:

Найти решение конечношаговой позиционной игры с полной информацией. Для этого сгенерировать и построить дерево случайной игры согласно варианту, используя метод обратной индукции, найти решение игры и путь (все пути, если он не единственный) к этому решению. Обозначить их на дереве.

Исходные данные:

Глубина дерева – 5

Количество игроков – 3

Количество стратегий – [2, 4, 2]

Диапазон выигрышей – [-50, 20]

Ход работы:Распределение ходов по уровням дерева:

Глубина 0 – ходит игрок 1;

Глубина 1 – ходит игрок 2;

Глубина 2 – ходит игрок 3;

Глубина 3 – ходит игрок 1;

Глубина 4 – ходит игрок 2;

Глубина 5 – терминальные вершины, выбор не производится.

Размер дерева игры:

Всего вершин – 187;

Внутренних вершин – 59;

Листьев – 128.

Позиционная игра с полной информацией представляется в виде дерева.

Каждая внутренняя вершина соответствует ситуации выбора одного из игроков, а каждая терминальная вершина содержит вектор выигрышей всех игроков.

Метод обратной индукции заключается в последовательной обработке дерева, начиная с предпоследнего уровня:

1. на текущем уровне рассматриваются все дочерние вершины;
2. выбираются те исходы, которые максимизируют выигрыш игрока, ходящего в данной вершине;
3. если несколько исходов дают одинаковый максимум, сохраняются все такие варианты;
4. затем обработанные значения передаются выше по дереву;
5. процесс продолжается до корня.

В результате в корневой вершине получается значение игры и один или несколько оптимальных путей.

По заданным параметрам было сформировано полное дерево глубины 5.

На каждом уровне игроки делают ходы в соответствии с циклическим порядком: $1 \rightarrow 2 \rightarrow 3 \rightarrow 1 \rightarrow 2$.

В терминальных вершинах были заданы векторы выигрышей трёх игроков из диапазона $[-50; 20]$.

Чтобы не загромождать рисунки, обозначения игроков у каждого узла не приводятся, так как нетрудно определить, кто ходит в каждом узле, зная, чей ход первый. Но для удобства на каждом рисунке есть строка с порядком ходов.

На рисунке 1 показана структура дерева исходной игры без последнего уровня. Терминальные вершины не приведены, поскольку полное дерево содержит 128 листьев и неудобно для размещения в отчёте. В полном дереве из каждой вершины, которая на данном рисунке является терминальной, выходит по 4 листа.

Полное дерево без терминальных вершин
 Порядок ходов $1 \rightarrow 2 \rightarrow 3 \rightarrow 1 \rightarrow 2$

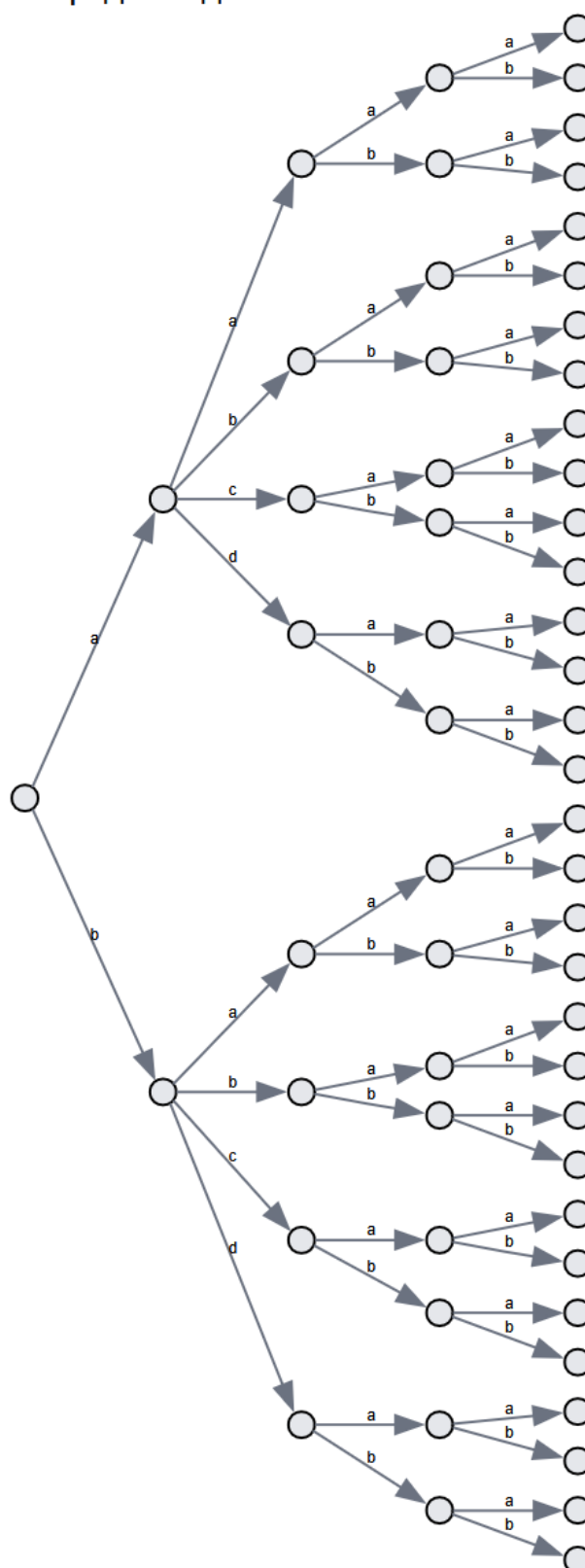


Рисунок 1 – Полное дерево без терминальных вершин

Дальше пошагово обрабатывается каждый уровень.

На уровне 4 ход делает игрок 2.

В каждой вершине выбирались те терминальные исходы, для которых вторая компонента максимальна.

В результате были получены, следующие значения:

- v5: значение (19, 19, -12)
- v10: значение (-7, 15, -8)
- v16: значение (-32, 10, -16)
- v21: значение (-34, 13, -15)
- v28: значение (16, 0, -10)
- v33: значение (-26, 19, 3)
- v39: значение (-49, 17, 3)
- v44: значение (15, 19, -5)
- v51: значение (-17, -8, -18)
- v56: значение (7, 14, -28)
- v62: значение (-9, 2, -32)
- v67: значение (15, 9, -22)
- v74: значение (-47, -15, -17)
- v79: значение (-5, 0, 3)
- v85: значение (-38, -9, -13)
- v90: значение (-12, -9, -45)
- v98: значение (18, 5, -17)
- v103: значение (18, 7, -13)
- v109: значение (-2, 17, -22)
- v114: значение (-3, -12, -35)
- v121: значение (-2, -10, -33)
- v126: значение (-37, 5, -17)
- v132: значение (9, -20, -46)

v137: значение (14, 2, 12)

v144: значение (2, 20, -22)

v149: значение (0, 0, 19)

v155: несколько равнозначных исходов $[(-1, 14, -16), (3, 14, 16)]$

v160: значение (-36, 17, -4)

v167: значение (17, 18, -6)

v172: значение (19, 18, 13)

v178: несколько равнозначных исходов $[(-33, -16, 14), (15, -16, -50)]$

v183: значение (-47, 11, 2)

На этом уровне встретились вершины с несколькими равнозначными исходами. Это означает, что в данных вершинах игрок 2 имеет несколько равноценных для себя продолжений.

Первый шаг алгоритма графически не приводится, так как соответствующий рисунок получается слишком большим и плохо читаемым.

На уровне 3 ход делает игрок 1.

Сравнение выполнялось по первой компоненте выигрыша.

Получены следующие значения:

v4: значение (19, 19, -12)

v15: значение (-32, 10, -16)

v27: значение (16, 0, -10)

v38: значение (15, 19, -5)

v50: значение (7, 14, -28)

v61: значение (15, 9, -22)

v73: значение (-5, 0, 3)

v84: значение (-12, -9, -45)

v97: несколько равнозначных исходов $[(18, 5, -17), (18, 7, -13)]$

v108: значение (-2, 17, -22)

v120: значение (-2, -10, -33)

v131: значение (14, 2, 12)

v143: значение (2, 20, -22)

v154: несколько равнозначных исходов [(-1, 14, -16), (3, 14, 16)]

v166: значение (19, 18, 13)

v177: несколько равнозначных исходов [(-33, -16, 14), (15, -16, -50)]

На уровне 2 ход делает игрок 3.

Сравнение выполнялось по третьей компоненте выигрыша.

Получены значения:

v3: значение (19, 19, -12)

v26: значение (15, 19, -5)

v49: значение (15, 9, -22)

v72: значение (-5, 0, 3)

v96: несколько равнозначных исходов [(18, 5, -17), (18, 7, -13)]

v119: значение (14, 2, 12)

v142: несколько равнозначных исходов [(-1, 14, -16), (3, 14, 16)]

v165: несколько равнозначных исходов [(-33, -16, 14), (19, 18, 13)]

На уровне 1 ход делает игрок 2.

Получены значения:

v2: несколько исходов [(15, 19, -5), (19, 19, -12)]

v95: несколько исходов [(-1, 14, -16), (3, 14, 16), (19, 18, 13)]

Это означает, что в обеих вершинах уровня 1 после учёта продолжений нижних подыгр сохраняется несколько возможных исходов.

На уровне 0 ход делает игрок 1.

В корне были получены три итоговых исхода:

v1: несколько исходов [(15, 19, -5), (19, 18, 13), (19, 19, -12)]

Таким образом решение игры в текущей реализации оказалось неединственным (рисунок 2):

```
Оптимальный путь #1
Выигрыши: (15, 19, -5)
Путь по вершинам: [1, 2, 26, 38, 44, 48]
v1 --a--> v2
v2 --b--> v26
v26 --b--> v38
v38 --b--> v44
v44 --d--> v48

Оптимальный путь #2
Выигрыши: (19, 18, 13)
Путь по вершинам: [1, 95, 165, 166, 172, 173]
v1 --b--> v95
v95 --d--> v165
v165 --a--> v166
v166 --b--> v172
v172 --a--> v173

Оптимальный путь #3
Выигрыши: (19, 19, -12)
Путь по вершинам: [1, 2, 3, 4, 5, 6]
v1 --a--> v2
v2 --a--> v3
v3 --a--> v4
v4 --a--> v5
v5 --a--> v6
```

Рисунок 2 – Оптимальные пути найденные методом обратной индукции

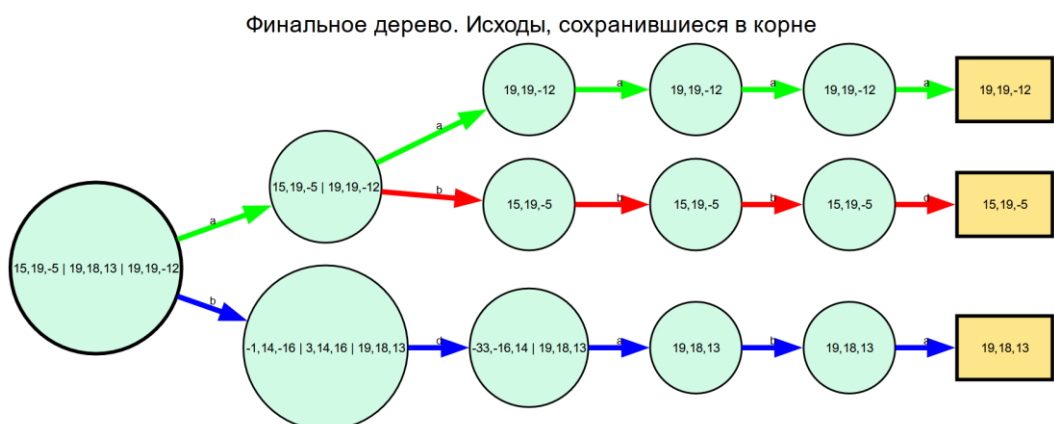


Рисунок 3 – Визуализация найденных оптимальных путей

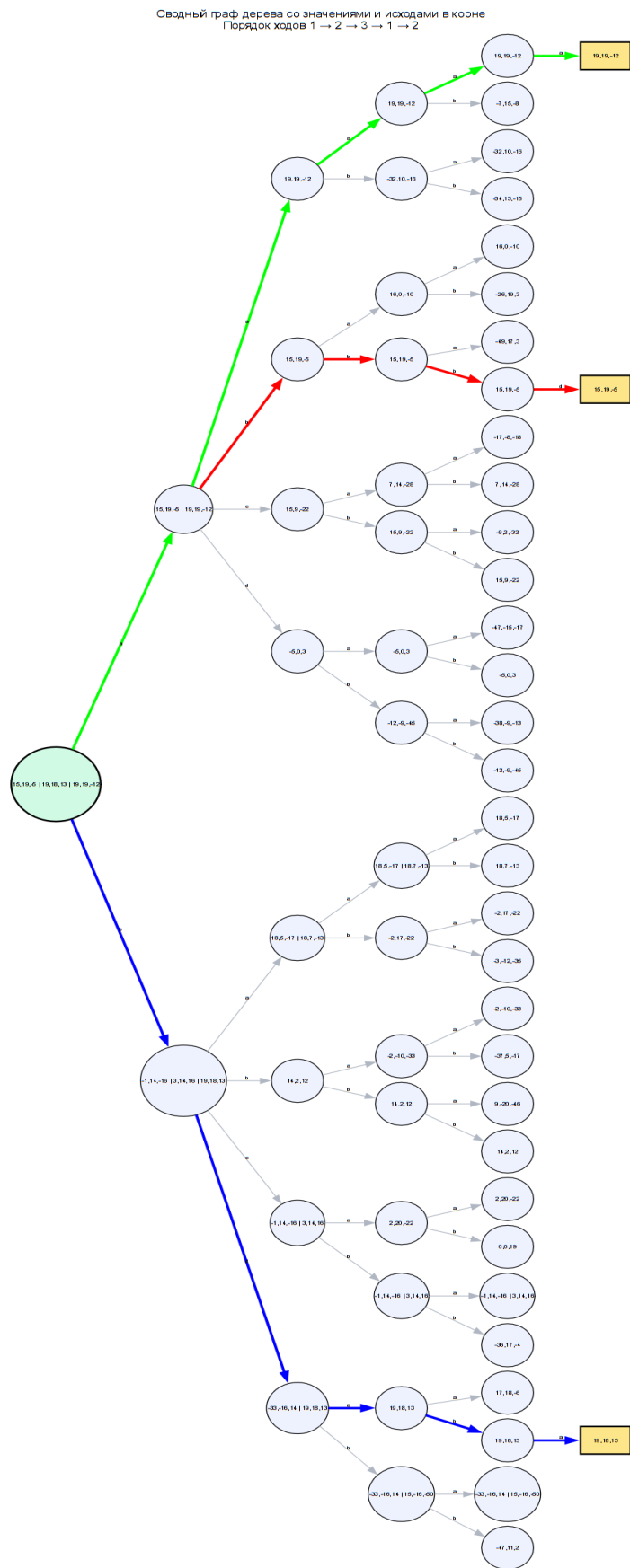


Рисунок 4 – Оптимальные пути на дереве позиционной игры

Для наглядного представления решения удобно использовать итоговый граф, на котором:

- подписаны внутренние вершины;
- показаны итоговые значения;
- выделены найденные пути.

На рисунке 4 приведено итоговое дерево со значениями, полученными методом обратной индукции. Цветом выделены итоговые пути, соответствующие исходам, сохранившимся в корне.

Исходя из того, что граф (даже без терминальных вершин) достаточно большой, и в полном размере может плохо читаться, на рисунке 3 приведены только найденные оптимальные пути без вершин, не участвующих в этих путях.

Вывод:

В ходе лабораторной работы было построено дерево позиционной игры и выполнено решение методом обратной индукции.

В результате:

- построено дерево глубины 5;
- определены значения во всех внутренних вершинах;
- учтены случаи равнозначных продолжений;

В корневой вершине получены три итоговых исхода:

$$(15, 19, -5), \quad (19, 18, 13), \quad (19, 19, -12).$$

Таким образом, решение игры в текущем варианте оказалось неединственным, а обратная индукция позволила последовательно получить все исходы, сохраняющиеся в корне после обработки всех уровней дерева.

Приложение (исходный код)

```
from __future__ import annotations

from dataclasses import dataclass, field
from typing import Dict, List, Optional, Tuple, Set
import os
import random

from itertools import product
from graphviz import Digraph

VARIANT = 13
TREE_DEPTH = 5
NUM_PLAYERS = 3
STRATEGIES_PER_PLAYER = [2, 4, 2]
PAYOFF_MIN = -50
PAYOFF_MAX = 20
RANDOM_SEED = 56578

OUTPUT_DIR = "data"

GRAPH_FORMAT = "pdf" # Либо "svg"
GRAPH_ENGINE = "dot"

SAVE_FULL_TREE = True
SAVE_FULL_TREE_WITHOUT_LEAVES = True
SAVE_STEP_GRAPHS = True
SAVE_FINAL_GRAPH = True
SAVE_SUMMARY_GRAPH = True
SAVE_ALL_RATIONAL_GRAPH = True

PRINT_TERMINAL_PAYOFFS = False
PRINT_STEP_SUMMARY = True
PRINT_DETAILED_STEP_INFO = False

@dataclass(frozen=True)
class Candidate:
    payoff: Tuple[int, ...]
    path: Tuple[int, ...]

@dataclass
class Node:
    id: int
    depth: int
    player: Optional[int]
    parent_id: Optional[int]
    action_from_parent: Optional[str]
    children_ids: List[int] = field(default_factory=list)
    terminal_payoff: Optional[Tuple[int, ...]] = None
    resolved_candidates: List[Candidate] = field(default_factory=list)

    @property
    def is_leaf(self) -> bool:
        return self.player is None

    def name(self) -> str:
        return f"v{self.id}"

class PositionalGame:
    def __init__(
        self,
        depth: int,
        num_players: int,
        strategies_per_player: List[int],
        payoff_min: int,
        payoff_max: int,
        seed: int,
    ) -> None:
        self.depth = depth
        self.num_players = num_players
        self.strategies_per_player = strategies_per_player
```

```

        self.payoff_min = payoff_min
        self.payoff_max = payoff_max
        self.seed = seed

        self.nodes: Dict[int, Node] = {}
        self.root_id: Optional[int] = None
        self._next_id = 1
        self.rng = random.Random(seed)

    def player_for_depth(self, depth: int) -> Optional[int]:
        if depth >= self.depth:
            return None
        return (depth % self.num_players) + 1

    def branching_for_player(self, player: int) -> int:
        return self.strategies_per_player[player - 1]

    def action_labels_for_player(self, player: int) -> List[str]:
        return [chr(ord("a") + i) for i in range(self.branching_for_player(player))]

    def random_payoff(self) -> Tuple[int, ...]:
        return tuple(
            self.rng.randint(self.payoff_min, self.payoff_max)
            for _ in range(self.num_players)
        )

    def root(self) -> Node:
        assert self.root_id is not None
        return self.nodes[self.root_id]

    def nodes_at_depth(self, depth: int) -> List[Node]:
        arr = [n for n in self.nodes.values() if n.depth == depth]
        arr.sort(key=lambda x: x.id)
        return arr

    def leaves(self) -> List[Node]:
        arr = [n for n in self.nodes.values() if n.is_leaf]
        arr.sort(key=lambda x: x.id)
        return arr

    def internal_nodes(self) -> List[Node]:
        arr = [n for n in self.nodes.values() if not n.is_leaf]
        arr.sort(key=lambda x: x.id)
        return arr

    def unique_payoffs_at_node(self, node: Node) -> List[Tuple[int, ...]]:
        return sorted(set(c.payoff for c in node.resolved_candidates))

    def path_to_actions(self, path: Tuple[int, ...]) -> List[Tuple[int, str, int]]:
        result = []
        for i in range(len(path) - 1):
            parent = self.nodes[path[i]]
            child = self.nodes[path[i + 1]]
            result.append((parent.id, child.action_from_parent or "?", child.id))
        return result

    def _compact_payoff_label(self, payoff: Tuple[int, ...]) -> str:
        return ",".join(str(x) for x in payoff)

    def _compact_levels_label(self) -> str:
        return "Порядок ходов 1 → 2 → 3 → 1 → 2"

    def build_full_tree(self) -> None:
        self.nodes.clear()
        self._next_id = 1
        self.root_id = self._build_subtree(0, None, None)

    def _build_subtree(
        self,
        depth: int,
        parent_id: Optional[int],
        action_from_parent: Optional[str],
    ) -> int:
        node_id = self._next_id
        self._next_id += 1

```

```

player = self.player_for_depth(depth)

node = Node(
    id=node_id,
    depth=depth,
    player=player,
    parent_id=parent_id,
    action_from_parent=action_from_parent,
)
self.nodes[node_id] = node

if player is None:
    node.terminal_payoff = self.random_payoff()
    node.resolved_candidates = [Candidate(node.terminal_payoff, (node.id,))]
    return node_id

for action in self.action_labels_for_player(player):
    child_id = self._build_subtree(depth + 1, node_id, action)
    node.children_ids.append(child_id)

return node_id

def _resolve_node(self, node: Node) -> None:
    assert not node.is_leaf
    assert node.player is not None

    mover_idx = node.player - 1

    child_candidate_lists: List[List[Candidate]] = []
    for child_id in node.children_ids:
        child = self.nodes[child_id]

        prefixed_candidates = [
            Candidate(
                payoff=cand.payoff,
                path=(node.id,) + cand.path,
            )
            for cand in child.resolved_candidates
        ]
        child_candidate_lists.append(prefixed_candidates)

    result: Dict[Tuple[Tuple[int, ...], Tuple[int, ...]], Candidate] = {}

    for combo in product(*child_candidate_lists):
        best_value = max(c.payoff[mover_idx] for c in combo)

        for cand in combo:
            if cand.payoff[mover_idx] == best_value:
                key = (cand.payoff, cand.path)
                result[key] = cand

    unique = list(result.values())
    unique.sort(key=lambda c: (c.payoff, c.path))
    node.resolved_candidates = unique

def print_summary(self) -> None:
    print("=" * 100)
    print("ЛР №4 (по методичке ЛР №6) — ПОЗИЦИОННЫЕ ИГРЫ")
    print("=" * 100)
    print(f"Вариант: {VARIANT}")
    print(f"Seed генератора: {self.seed}")
    print()
    print("Параметры варианта:")
    print(f"  Глубина дерева: {self.depth}")
    print(f"  Количество игроков: {self.num_players}")
    print(f"  Количество стратегий игроков: {self.strategies_per_player}")
    print(f"  Диапазон выигрышей: [{self.payoff_min}, {self.payoff_max}]")
    print()
    print("Кто ходит на каждом уровне:")
    for d in range(self.depth):
        p = self.player_for_depth(d)
        print(f"  Глубина {d}: ходит игрок {p}, число альтернатив = {self.branching_for_player(p)}")
    print(f"  Глубина {self.depth}: листья, выбор никто не делает")
    print()
    print(f"Всего вершин: {len(self.nodes)}")

```

```

print(f"Внутренних вершин: {len(self.internal_nodes())}")
print(f"Листьев: {len(self.leaves())}")
print("=" * 100)

def print_terminal_payoffs(self) -> None:
    print("\nТЕРМИНАЛЬНЫЕ ВЕРШИНЫ И ИХ ВЫИГРЫШИ")
    print("=" * 100)
    for node in self.leaves():
        print(f"{node.name():>4} depth={node.depth} payoff={node.terminal_payoff}")

def print_step_summary(self, current_depth: int) -> None:
    player = self.player_for_depth(current_depth)
    print()
    print("=" * 100)
    print(f"УРОВЕНЬ {current_depth} – ходит игрок {player}")
    print("=" * 100)
    for node in self.nodes_at_depth(current_depth):
        payoffs = self.unique_payoffs_at_node(node)
        if len(payoffs) == 1:
            print(f"{node.name():>4}: значение {payoffs[0]}")
        else:
            print(f"{node.name():>4}: несколько равнозначных исходов {payoffs}")

def print_detailed_step_info(self, current_depth: int) -> None:
    player = self.player_for_depth(current_depth)
    print()
    print("=" * 100)
    print(f"ИТЕРАЦИЯ ОБРАТНОЙ ИНДУКЦИИ: ОБРАБАТЫВАЕМ УРОВЕНЬ {current_depth}")
    print(f"На этом уровне ходит игрок {player}")
    print("=" * 100)

    for node in self.nodes_at_depth(current_depth):
        assert node.player is not None
        mover_idx = node.player - 1

        print(f"\nВершина {node.name()} (глубина {node.depth}, ходит игрок {node.player})")
        print("Дочерние вершины и доступные результаты:")

        rows = []
        for child_id in node.children_ids:
            child = self.nodes[child_id]
            for cand in child.resolved_candidates:
                rows.append((child.id, child.action_from_parent or "?", cand.payoff,
cand.payoff[mover_idx]))

        rows.sort(key=lambda x: (x[0], x[2], x[1]))
        for child_id, action, payoff, mover_value in rows:
            print(
                f"    -> {node.name()} --{action}--> v{child_id}: "
                f"payoff={payoff}, компонент игрока {node.player} = {mover_value}"
            )

        best_value = max(x[3] for x in rows)
        print(f"Лучшее значение для игрока {node.player}: {best_value}")
        print("Оставляем в узле:")
        for cand in node.resolved_candidates:
            print(f"    payoff={cand.payoff}, path={list(cand.path)}")

def print_final_solutions(self) -> None:
    root = self.root()

    print()
    print("=" * 100)
    print("ИТОГОВОЕ РЕШЕНИЕ ИГРЫ")
    print("=" * 100)

    print("Значение игры в корне:")
    for payoff in self.unique_payoffs_at_node(root):
        print(f"    {payoff}")

    print()
    print(f"Количество оптимальных исходов в корне: {len(root.resolved_candidates)}")
    print()

    for idx, cand in enumerate(root.resolved_candidates, start=1):
        print(f"Оптимальный путь #{idx}")

```

```

        print(f"  Выигрыши: {cand.payoff}")
        print(f"  Путь по вершинам: {list(cand.path)}")
        for u, act, v in self.path_to_actions(cand.path):
            print(f"    v{u} --{act}--> v{v}")
        print()

def collect_root_optimal_edges(self) -> Dict[Tuple[int, int], str]:
    root = self.root()
    final_paths = [cand.path for cand in root.resolved_candidates]

    colors = [
        "red", "blue", "green", "orange", "purple",
        "brown", "magenta", "cyan", "darkgreen", "darkgoldenrod"
    ]

    edge_to_color: Dict[Tuple[int, int], str] = {}
    for i, path in enumerate(final_paths):
        color = colors[i % len(colors)]
        for j in range(len(path) - 1):
            edge_to_color[(path[j], path[j + 1])] = color
    return edge_to_color

def collect_all_rational_edges(self) -> Set[Tuple[int, int]]:
    edges: Set[Tuple[int, int]] = set()
    for node in self.internal_nodes():
        for cand in node.resolved_candidates:
            path = cand.path
            for i in range(len(path) - 1):
                edges.add((path[i], path[i + 1]))
    return edges

def collect_all_rational_leaves(self) -> Set[int]:
    leaves: Set[int] = set()
    for node in self.internal_nodes():
        for cand in node.resolved_candidates:
            leaf_id = cand.path[-1]
            if self.nodes[leaf_id].is_leaf:
                leaves.add(leaf_id)
    return leaves

def _make_graph(
    self,
    title: str,
    compact_levels_note: bool = False,
    a4_landscape: bool = True,
) -> Digraph:
    g = Digraph(format=GRAPH_FORMAT, engine=GRAPH_ENGINE)
    attrs = {
        "rankdir": "LR",
        "splines": "polyline",
        "overlap": "false",
        "nodesep": "0.08",
        "ranksep": "0.55",
        "pad": "0.04",
        "margin": "0.02",
        "concentrate": "true",
    }
    if a4_landscape:
        attrs["size"] = "11.69,8.27"
        attrs["ratio"] = "compress"
    g.attr(**attrs)

    label = f"{title}\\n{self._compact_levels_label()}" if compact_levels_note else title
    g.attr(label=label, labelloc="t", fontsize="13", fontname="Arial")
    g.attr("node", fontname="Arial", fontsize="7", margin="0.01,0.01")
    g.attr("edge", fontname="Arial", fontsize="6")
    return g

def _make_step_graph(self, title: str) -> Digraph:
    g = Digraph(format=GRAPH_FORMAT, engine=GRAPH_ENGINE)
    g.attr(
        rankdir="LR",
        splines="polyline",
        overlap="false",
        nodesep="0.18",
        ranksep="0.45",

```

```

        pad="0.05",
        margin="0.02",
        concentrate="true",
        size="11.69,8.27",
        ratio="compress",
    )
    g.attr(label=title, labelloc="t", fontsize="13", fontname="Arial")
    g.attr("node", fontname="Arial", fontsize="7", margin="0.01,0.01")
    g.attr("edge", fontname="Arial", fontsize="6")
    return g

def _node_label_value(self, node: Node) -> str:
    if node.is_leaf:
        return self._compact_payoff_label(node.terminal_payoff)

    vals = self.unique_payoffs_at_node(node)
    if len(vals) == 1:
        return self._compact_payoff_label(vals[0])
    if len(vals) > 1:
        return " | ".join(self._compact_payoff_label(v) for v in vals)
    return ""

def render_full_tree_compact(self, filename_no_ext: str) -> None:
    g = self._make_graph("Полное дерево игры", compact_levels_note=True)

    for nid in sorted(self.nodes.keys()):
        node = self.nodes[nid]

        if node.is_leaf:
            g.node(
                node.name(),
                label=self._compact_payoff_label(node.terminal_payoff),
                shape="box",
                style="filled",
                fillcolor="#FDE68A",
                width="0.24",
                height="0.10",
                fixedsize="false",
            )
        else:
            g.node(
                node.name(),
                label="",
                shape="circle",
                style="filled",
                fillcolor="#E5E7EB",
                width="0.12",
                height="0.12",
                fixedsize="true",
            )

    for node in self.nodes.values():
        for child_id in node.children_ids:
            child = self.nodes[child_id]
            g.edge(node.name(), child.name(), label=child.action_from_parent or "",
color="#6B7280")

    g.render(os.path.join(OUTPUT_DIR, filename_no_ext), cleanup=True)

def render_full_tree_without_leaves(self, filename_no_ext: str) -> None:
    g = self._make_graph("Полное дерево без терминальных вершин", compact_levels_note=True)

    internal_nodes = self.internal_nodes()

    for node in internal_nodes:
        g.node(
            node.name(),
            label="",
            shape="circle",
            style="filled",
            fillcolor="#E5E7EB",
            width="0.14",
            height="0.14",
            fixedsize="true",
        )

```

```

        for node in internal_nodes:
            for child_id in node.children_ids:
                child = self.nodes[child_id]
                if child.is_leaf:
                    continue
                g.edge(node.name(), child.name(), label=child.action_from_parent or "",
color="#6B7280")

        g.render(os.path.join(OUTPUT_DIR, filename_no_ext), cleanup=True)

def render_local_step_graph(self, current_depth: int, filename_no_ext: str) -> None:
    title = f"Уровень {current_depth} – ходит игрок {self.player_for_depth(current_depth)}"
    g = self._make_step_graph(title)

    current_nodes = self.nodes_at_depth(current_depth)
    g.node("LEVEL_ROOT", label=f"Уровень {current_depth}", shape="plaintext")

    for node in current_nodes:
        node_name = node.name()

        g.node(
            node_name,
            label=self._node_label_value(node),
            shape="circle",
            style="filled",
            fillcolor="#D1FAE5",
            penwidth="2",
            width="0.60",
        )
        g.edge("LEVEL_ROOT", node_name, color="#666666")

        for child_id in node.children_ids:
            child = self.nodes[child_id]
            child_name = child.name()

            if child.is_leaf:
                g.node(
                    child_name,
                    label=self._node_label_value(child),
                    shape="box",
                    style="filled",
                    fillcolor="#FDE68A",
                )
            else:
                g.node(
                    child_name,
                    label=self._node_label_value(child),
                    shape="circle",
                    style="filled",
                    fillcolor="#EEF2FF",
                    width="0.60",
                )

            g.edge(node_name, child_name, label=child.action_from_parent or "")

        g.render(os.path.join(OUTPUT_DIR, filename_no_ext), cleanup=True)

def render_final_graph(self, filename_no_ext: str) -> None:
    g = self._make_step_graph("Финальное дерево. Исходы, сохранившиеся в корне")

    edge_to_color = self.collect_root_optimal_edges()
    node_ids: Set[int] = set()
    for (u, v) in edge_to_color.keys():
        node_ids.add(u)
        node_ids.add(v)

    for nid in sorted(node_ids):
        node = self.nodes[nid]
        if node.is_leaf:
            g.node(
                node.name(),
                label=self._node_label_value(node),
                shape="box",
                style="filled",
                fillcolor="#FDE68A",
                penwidth="2",
            )

```

```

    )
    else:
        g.node(
            node.name(),
            label=self._node_label_value(node),
            shape="circle",
            style="filled",
            fillcolor="#D1FAE5",
            penwidth="2" if nid == self.root_id else "1",
            width="0.72",
        )

for (u, v), color in edge_to_color.items():
    child = self.nodes[v]
    g.edge(
        self.nodes[u].name(),
        child.name(),
        label=child.action_from_parent or "",
        color=color,
        penwidth="3",
    )

g.render(os.path.join(OUTPUT_DIR, filename_no_ext), cleanup=True)

def render_all_rational_graph(self, filename_no_ext: str) -> None:
    g = self._make_graph(
        "Все локально рациональные продолжения подыгр",
        compact_levels_note=True,
        a4_landscape=True,
    )

    all_rational_edges = self.collect_all_rational_edges()
    root_optimal_edges = self.collect_root_optimal_edges()
    rational_leaves = self.collect_all_rational_leaves()

    for node in self.internal_nodes():
        g.node(
            node.name(),
            label=self._node_label_value(node),
            shape="circle",
            style="filled",
            fillcolor="#EEF2FF" if node.id != self.root_id else "#D1FAE5",
            penwidth="2" if node.id == self.root_id else "1",
            width="0.78",
        )

    for leaf_id in sorted(rational_leaves):
        leaf = self.nodes[leaf_id]
        g.node(
            leaf.name(),
            label=self._node_label_value(leaf),
            shape="box",
            style="filled",
            fillcolor="#FDE68A",
            penwidth="1",
        )

    for node in self.internal_nodes():
        for child_id in node.children_ids:
            child = self.nodes[child_id]
            edge = (node.id, child_id)

            if child.is_leaf and child_id not in rational_leaves:
                continue

            if edge not in all_rational_edges and edge not in root_optimal_edges:
                continue

            if edge in root_optimal_edges:
                color = root_optimal_edges[edge]
                penwidth = "3"
                style = "solid"
            else:
                color = "#1F4E79"
                penwidth = "1.5"
                style = "dashed"

```

```

        g.edge(
            node.name(),
            child.name(),
            label=child.action_from_parent or "",
            color=color,
            penwidth=penwidth,
            style=style,
        )

    g.render(os.path.join(OUTPUT_DIR, filename_no_ext), cleanup=True)

def render_summary_value_graph(self, filename_no_ext: str) -> None:
    g = self._make_graph(
        "Сводный граф дерева со значениями и исходами в корне",
        compact_levels_note=True,
        a4_landscape=True,
    )

    root_optimal_edges = self.collect_root_optimal_edges()
    root_optimal_leaves = {v for (_, v) in root_optimal_edges if self.nodes[v].is_leaf}

    for node in self.internal_nodes():
        g.node(
            node.name(),
            label=self._node_label_value(node),
            shape="circle",
            style="filled",
            fillcolor="#D1FAE5" if node.id == self.root_id else "#EEF2FF",
            penwidth="2" if node.id == self.root_id else "1",
            width="0.78",
        )

    for leaf_id in sorted(root_optimal_leaves):
        leaf = self.nodes[leaf_id]
        g.node(
            leaf.name(),
            label=self._node_label_value(leaf),
            shape="box",
            style="filled",
            fillcolor="#FDE68A",
            penwidth="2",
        )

    for node in self.internal_nodes():
        for child_id in node.children_ids:
            child = self.nodes[child_id]

            if child.is_leaf and child_id not in root_optimal_leaves:
                continue

            edge = (node.id, child_id)
            if edge in root_optimal_edges:
                color = root_optimal_edges[edge]
                penwidth = "3"
            else:
                color = "#B0B7C3"
                penwidth = "1"

            g.edge(
                node.name(),
                child.name(),
                label=child.action_from_parent or "",
                color=color,
                penwidth=penwidth,
            )

    g.render(os.path.join(OUTPUT_DIR, filename_no_ext), cleanup=True)

def ensure_output_dir(path: str) -> None:
    os.makedirs(path, exist_ok=True)

def main() -> None:
    ensure_output_dir(OUTPUT_DIR)

```

```

game = PositionalGame(
    depth=TREE_DEPTH,
    num_players=NUM_PLAYERS,
    strategies_per_player=STRATEGIES_PER_PLAYER,
    payoff_min=PAYOFF_MIN,
    payoff_max=PAYOFF_MAX,
    seed=RANDOM_SEED,
)

game.build_full_tree()

if SAVE_FULL_TREE:
    game.render_full_tree_compact("full_tree_compact")

if SAVE_FULL_TREE_WITHOUT_LEAVES:
    game.render_full_tree_without_leaves("full_tree_without_leaves")

game.print_summary()

if PRINT_TERMINAL_PAYOFFS:
    game.print_terminal_payoffs()

for current_depth in range(game.depth - 1, -1, -1):
    for node in game.nodes at depth(current_depth):
        game._resolve_node(node)

        if PRINT_STEP_SUMMARY:
            game.print_step_summary(current_depth)

        if PRINT_DETAILED_STEP_INFO:
            game.print_detailed_step_info(current_depth)

        if SAVE_STEP_GRAPHS:
            game.render_local_step_graph(
                current_depth=current_depth,
                filename_no_ext=f"step {game.depth -
current_depth:02d}_after_depth_{current_depth}",
            )

game.print_final_solutions()

if SAVE_FINAL_GRAPH:
    game.render_final_graph("step_final_optimal_paths")

if SAVE_SUMMARY_GRAPH:
    game.render_summary_value_graph("summary_value_graph")

if SAVE_ALL_RATIONAL_GRAPH:
    game.render_all_rational_graph("all_rational_continuations_graph")

if __name__ == "__main__":
    main()

```